< Winston Industries LLC >

< Arithmetic Expression Evaluator in C++ >
Software Requirements Specifications

Version <1.0>

Revision History

Date	Version	Description	Author
<13/Mar/26>	0.1	Added content for 2.1-2.6	Davina Love
<13/Mar/26>	0.2	Added content for 1.4-1.5	Conner Magee
<13/Mar/26>	0.3	Added content for 1-1.1.3	Landrie Rolla
<14/Mar/26>	0.4	Added content for 3.1, 3.2, and 4	Liam Gunther
<14/March/26>	1.0	Version 1.0 Complete	Davina Love

©<Company Name>inston Industries LLC 2026

Table of Contents

1. Introduction 4
   1.1 Purpose 4
   1.2 Scope 4
   1.3 Definitions, Acronyms, and Abbreviations 4
   1.4 References 4
   1.5 Overview 4

2. Overall Description 5
   2.1 Product perspective 5
      2.1.1 System Interfaces 5
      2.1.2 User Interfaces 5
      2.1.3 Hardware Interfaces 5
      2.1.4 Software Interfaces 5
      2.1.5 Communication Interfaces 5
      2.1.6 Memory Constraints 5
      2.1.7 Operations 5
   2.2 Product functions 5
   2.3 User characteristics 5
   2.4 Constraints 5
   2.5 Assumptions and dependencies 5
   2.6 Requirements subsets 5

3. Specific Requirements 6
   3.1 Functionality 6
      3.1.1 Expression Parsing 6
      3.1.2 Operator Support and precedence 6
      3.1.3 Support Unary + and - 6
      3.1.4 Handle Parenthesis 6
      3.1.5 Interpret Numeric Constants 6
      3.1.6 Interpret numeric constants in floating-point 6
      3.1.7 Interpret [ ] as an additional option for grouping expressions 6
   3.2 Use-Case Specifications 6
   3.3 Supplementary Requirements 7

4. Classification of Functional Requirements 7

5. Appendices 8

Software Requirements Specifications

1. Introduction

The purpose of this document is to outline the development of a large-scale compiler in language L. The compiler should be able to tokenize the user input and perform actions.

1.1 Purpose

The Software Requirements Specification (SRS) describes the requirements for the Arithmetic Expression Parser and Evaluator components of a larger compiler system for language L. The purpose of this document is to provide a description of the system's functional and non-function requirements, design constraints and external behavior.

1.2 Scope

The system will be able to accept arithmetic expressions from the user through a command-line interface, parse the expressions, and compute the correct results based on operator precedence and associativity rules (PEMDAS). The system will support the following operators:

- Addition
- Subtraction
- Multiplication
- Division
- Modulo
- Exponentiation
- Unary operators
- Parenthesis
- Integers

1.3 Definitions, Acronyms, and Abbreviations

Parser – A program component that analyses input according to grammar and determines its structure.

Token- A meaningful sequence of characters in an expression.

Expression- A combination of operators and operands that can be evaluated to produce a value.

Associativity- Rules that determine evaluation order when operators have the same precedence.

PEMDAS- The order of operations, parenthesis, exponents, multiplication, division, addition, subtraction.

1.4 References

N/A

1.5 Overview

This document details the technical and non-technical specifications of the project.

Section two describes the technical requirements and specifications of the project. This includes software

Section three

Section four

2. Overall Description

2.1 Product perspective

2.1.1 System Interfaces

This program will be using a terminal interface for user input and displaying the output.

2.1.2 User Interfaces

The user interface will be a terminal window formatted in a way that is easy-to-read and allows for easy selection/input. If the user inputs something that is unable be parsed, the terminal will display a descriptive error message and allow the user to retry.

2.1.3 Hardware Interfaces

Input from the user's keyboard will be parsed.

2.1.4 Software Interfaces

The program will run on the user's choice of terminal. Examples include Windows cmd, PowerShell, Git Bash, etc.

2.1.5 Communication Interfaces

The Arithmetic Expression Evaluator will be run locally on the user's computer.

2.1.6 Memory Constraints

The program will be designed to be lean enough to where the average user's machine will be able to run the program. Operations
The Arithmetic Expression Evaluator will be able to parse user keyboard input (integers, floating point, and operators) and be able to perform basic operations (addition, subtraction, multiplication, division, modulo, exponential) and output the result in a readable format to the console.

2.2 Product functions

The product should be able to correctly parse user input and perform simple arithmetic calculations.

2.3 User characteristics

The product will accept input in a command-line interface as a way for the user to navigate through the program. The output will be displayed in the terminal.

2.4 Constraints

The first iteration of the product will only include support for integer numbers. Later iterations may possibly include floating point support. The range of integer numbers will be -2^16 to 2^16. (as of ver 1.0)

The user will also not be able to do invalid calculations such as division by zero.

2.5 Assumptions and dependencies

In order to run this program, it is assumed that the user will have access to some existing command line interface such as Windows cmd or Git Bash.

2.6 Requirements subsets

Parse User Input	
Recognize integers	Recognize operators (+, -, / , *, %, **, (), []) and

Unary Operators: (++, --, !, -, +)	perform operations in PEMDAS order
Evaluate Expressions	
Addition	Subtraction
Multiplication	Division
Modulo	Exponents
Error-Handling	
Integer limit	Float point rounding
Invalid input (eg. Alphanumeric characters, special characters)	Ignore spacing

3. Specific Requirements

3.1 Functionality

3.1.1 Expression Parsing

FR1: The system will use a function to tokenize an expression given by the user from a terminal interface.
FR2: The system uses a stack to keep the structure of the expression.

3.1.2 Operator Support and precedence

FR3: The systems parser and evaluator will be able to parse and correctly calculate an expression containing each of the operators (+, -, / , *, %, **). (exponentiation (**) is read right to left
FR4: Expressions will be parsed using precedence directly according to the rules of PEMDAS.

3.1.3 Support Unary + and -

FR5: The evaluator will parse the unary operators + or – to evaluate negative values or to indicate a possibly positive value.

3.1.4 Handle Parenthesis

FR6: The system's Parser will accept parentheses as a grouping expression at the top of operator precedence.

3.1.5 Interpret Numeric Constants

FR7: The system will accept numeric constants in the terminal input, assumed to be integer type before potential future changes.

3.1.6 Interpret numeric constants in floating-point

FR8: The data type of the parser input/output and expression evaluator input/output can be modified to float instead of the current int type.

3.1.7 Interpret [ ] as an additional option for grouping expressions

FR9: The systems parser can be modified to accept [ ] brackets as an additional grouping expression to hand to the evaluator.

3.2 Use-Case Specifications

Use Case: Perform Calculation

Description: Allows user to calculate integer arithmetic expressions
Pre-Condition: User executes the system main executable
Post-Condition: The System prints the outcome of the calculation to the user interface
Basic Path:
1. User inputs a valid expression into the terminal
2. Parser is called on input expression
3. Arithmetic Evaluator calculates the expression's result using the token from the Parser
4. Displays the calculated results to the interface
Exception Path:
1a. User Inputs Invalid expression
2a. System displays a descriptive error message
3a. System prompts the User to input a valid expression

3.3 Supplementary Requirements

NFR1 (Performance): The system shall handle and evaluate any valid expression of reasonable length (up to 1,000 characters) in under 1 second on a standard machine.

NFR2 (Reliability): Shall never crash on malformed input; robust error handling to manage scenarios like division by zero or invalid expressions and report the errors clearly.

NFR3 (Portability/Constraints): The system shall be implemented using C++ and shall compile without error using a standard C++ compiler (e.g. g++) on a Windows or Linux-based environment.

NFR4 (Maintainability): The system shall be implemented and compliant with C++ coding standards.

NFR5 (Maintainability): The codebase shall be organized into modules (e.g. tokenizer, parser, and evaluator) to facilitate maintainability and future expansion.

NFR6 (Testability): The system shall include a suite of rigorous test cases covering all valid and invalid inputs (e.g. division by zero or uneven parenthesis).

4. Classification of Functional Requirements

Functionality	Type
Expression Parsing (Tokenize input)	Essential
Operator support and Precedence	Essential
Support Unary + and -	Essential
Handle Parentheses (order of evaluation)	Essential
Interpret Numeric Constants	Essential
User Interface	Essential
Interpret numbers in floating point	Desirable

Interpret [ ] as an additional option for grouping expressions	Optional

5. Appendices

©<Company Name>inston Industries LLC 2026